

ESPRIT School of Business

Master 1 — Business Analytics (M1 BA)

Projet Intégré

RAPPORT DE PROJET

Analyse et Prédiction du Churn Client

Tuteur du projet : Aymen Ben Brik

Équipe projet : Nour Houda Sassi

Année universitaire : 2025 – 2026

Document confidentiel — Institution bancaire anonymisée

Table des matières

1. Résumé exécutif.....	4
2. Introduction et contexte.....	4
2.1 Contexte métier.....	4
2.2 Objectif du projet.....	5
2.3 Démarche et organisation.....	5
2.4 Structure du présent document.....	6
3. Description et exploration des données.....	6
3.1 Origine et volumétrie.....	6
3.2 Principales variables.....	6
3.3 Définition de la variable cible.....	6
3.4 Qualité des données et problèmes identifiés.....	7
4. Architecture de la solution.....	7
4.1 Vue d'ensemble.....	7
4.2 Justification des choix techniques.....	8
4.3 Flux de données de bout en bout.....	8
5. ETL et modélisation dimensionnelle.....	9
5.1 Pipeline ETL.....	9
5.1.1 Extraction (Extract).....	9
5.1.2 Transformation (Transform).....	9
5.1.3 Chargement (Load).....	9
5.2 Modèle dimensionnel.....	10
5.3 Contrôles de cohérence et de qualité.....	10
6. Business Intelligence et Power BI.....	10
6.1 Démarche.....	10
6.2 KPIs retenus.....	11
6.3 État d'avancement du rapport Power BI.....	11
6.4 Regroupement des catégories de compte.....	12
7. Machine Learning.....	12
7.1 Objectif et grain de la modélisation.....	12
7.2 Préparation des données et fuites de données identifiées.....	12
7.2.1 Construction du jeu de données.....	12
7.2.2 Fuites de données identifiées et corrigées.....	12
7.2.3 Encodage et prétraitement.....	13
7.2.4 Gestion du déséquilibre de classes.....	13

7.2.5 Séparation train / test.....	13
7.3 Modèle retenu.....	13
7.4 Rapport de classification détaillé.....	13
7.5 Interprétabilité.....	14
7.6 Sauvegarde et intégration du modèle.....	14
8. Interface web et déploiement.....	14
8.1 Objectifs et périmètre fonctionnel.....	14
8.2 Choix technique.....	14
8.3 Page d'accueil — KPIs de synthèse.....	14
8.4 Analyse du churn par segment.....	15
8.5 Prédiction individuelle et comptes à risque.....	15
8.6 Déploiement.....	15
9. Limites et perspectives.....	16
9.1 Limites méthodologiques.....	16
9.2 Limites d'ingénierie.....	16
9.3 Perspectives d'amélioration.....	16
10. Conclusion.....	17
Annexes.....	17
A. Glossaire.....	17
B. Références des outils utilisés.....	18

1. Résumé exécutif

La fidélisation de la clientèle est un enjeu stratégique majeur pour toute institution bancaire : le coût d'acquisition d'un nouveau client dépasse systématiquement celui de la conservation d'un client existant. Ce projet intégré, réalisé dans le cadre du Master 1 Business Analytics d'ESPRIT School of Business, avait pour objectif de concevoir, de bout en bout, une solution analytique complète permettant de comprendre, mesurer et anticiper le phénomène de churn (attrition) à partir d'un jeu de données bancaires réel et anonymisé.

La démarche adoptée a suivi la chaîne décisionnelle complète imposée par l'énoncé du projet, en respectant scrupuleusement la timeline de quatre semaines fournie par l'encadrant : mise en place d'un pipeline ETL (Extract – Transform – Load) reproductible, construction d'un entrepôt de données analytique (Data Warehouse) selon un modèle en étoile, définition et calcul des indicateurs métier (KPIs), construction de tableaux de bord Power BI, entraînement et évaluation de modèles de Machine Learning supervisés pour la prédiction du churn, puis développement et déploiement d'une application web permettant l'exploitation opérationnelle des résultats.

Sur le plan technique, l'équipe a retenu une stack cohérente et reproductible : Python (pandas) pour l'exploration et l'ETL, DuckDB comme entrepôt analytique léger, export au format Parquet pour l'alimentation de Power BI, XGBoost pour la modélisation prédictive (avec gestion explicite du déséquilibre de classes via `scale_pos_weight`), et Streamlit pour l'interface web, déployée gratuitement sur Streamlit Community Cloud.

Le modèle retenu permet d'estimer, pour chaque client, une probabilité de churn exploitable par les conseillers clientèle, à la fois via un formulaire de simulation individuelle et via une liste priorisée des comptes actifs à plus haut risque. Les tableaux de bord Power BI complètent ce dispositif en offrant une lecture agrégée du churn par segment (ancienneté, secteur d'activité, devise, catégorie de compte).

Ce rapport présente en détail chacune de ces étapes, les choix techniques effectués et leur justification, les difficultés rencontrées, notamment les risques de fuite de données identifiés lors de la préparation du jeu d'entraînement, ainsi que les limites du travail réalisé et les perspectives d'amélioration envisagées.

2. Introduction et contexte

2.1 Contexte métier

Une institution bancaire a mis à disposition de l'équipe projet un jeu de données réel et anonymisé décrivant ses clients, leurs comptes et les produits qu'ils détiennent. Dans le secteur bancaire, la rétention client constitue un enjeu économique de premier plan : la littérature professionnelle du secteur s'accorde à dire qu'acquérir un nouveau client coûte plusieurs fois plus cher que d'en conserver un existant. Anticiper le churn permet donc à la banque de cibler des actions de rétention

(offres personnalisées, contact proactif du conseiller, revue des conditions tarifaires) avant que le client ne se dirige vers la clôture de son compte.

Conformément aux règles de confidentialité imposées par le projet, ni le nom de l'institution, ni aucun élément permettant de l'identifier, ne sont mentionnés dans le présent document, dans le code source ou dans la présentation orale.

2.2 Objectif du projet

L'objectif du projet est de concevoir une solution analytique de bout en bout couvrant cinq composantes complémentaires :

- Un pipeline ETL qui extrait, nettoie et enrichit les données brutes ;
- Un entrepôt de données analytique modélisé en étoile, support des analyses de Business Intelligence ;
- Un ensemble de rapports Power BI restituant les KPIs métier de manière interactive ;
- Un modèle de Machine Learning supervisé capable de prédire la probabilité de churn d'un client ;
- Une application web exposant le modèle et les indicateurs à des utilisateurs non techniques, déployée publiquement.

Au-delà du livrable technique, le projet visait un objectif pédagogique : mettre en pratique, sur un cas réel, l'ensemble de la chaîne décisionnelle enseignée pendant le M1 Business Analytics — de la modélisation de données à la restitution décisionnelle — tout en travaillant selon une démarche projet structurée (gestion de versions, livrables intermédiaires, présentation orale).

2.3 Démarche et organisation

L'équipe, composée de 3 à 4 étudiants, a organisé son travail selon les quatre semaines imposées par la timeline officielle du projet, avec un point d'avancement hebdomadaire avec l'encadrant :

Semaine	Phase	Livrable de fin de semaine
S1	Cadrage et exploration	Pipeline ETL fonctionnel + rapport d'exploration
S2	Business Intelligence & Power BI	Data Warehouse + rapports Power BI
S3	Machine Learning	Modèle sauvegardé + notebook d'analyse
S4	Interface web, déploiement, livraison	Démo en ligne + rapport + présentation

Tableau 1 — Organisation du projet sur quatre semaines

Chaque membre de l'équipe a contribué à l'ensemble des composantes tout en se spécialisant progressivement sur un lot technique (ETL/DWH, BI/Power BI, Machine Learning, interface web), afin que chacun puisse présenter une partie de la solution lors de la soutenance finale. Le suivi du code a été assuré via un dépôt Git/GitHub avec des commits réguliers de chaque membre, conformément aux règles de versioning imposées.

2.4 Structure du présent document

Le présent rapport suit la logique de la chaîne décisionnelle mise en œuvre. La section 3 décrit et explore le jeu de données fourni. La section 4 présente l'architecture globale de la solution. La section 5 détaille le pipeline ETL et la modélisation dimensionnelle. La section 6 présente la démarche de Business Intelligence et les rapports Power BI. La section 7 est consacrée à la modélisation Machine Learning, de la préparation des données à l'interprétabilité du modèle retenu. La section 8 décrit l'interface web développée et son déploiement. La section 9 discute les limites du travail réalisé et les perspectives d'amélioration, avant la conclusion générale en section 10. Les annexes regroupent un glossaire, la structure du dépôt de code et les références des outils utilisés.

3. Description et exploration des données

3.1 Origine et volumétrie

Les données mises à disposition proviennent du système d'information d'une institution bancaire et ont été anonymisées avant leur remise à l'équipe : aucun identifiant personnel direct n'est exploitable, mais la structure et la cohérence métier réelle des données ont été conservées. Le jeu de données se compose de:

- Un fichier principal au format CSV, d'environ 130 Mo, comportant plusieurs centaines de milliers de lignes, où chaque ligne correspond à un couple (client, compte produit) — un même client pouvant apparaître plusieurs fois s'il détient plusieurs comptes ;
- Huit tables de dimensions au format Excel (.xlsx), de 6 à 80 Ko chacune, fournissant le libellé descriptif associé aux codes présents dans le fichier principal (industrie, devise, motif de clôture, catégorie de compte, etc.).

Cette volumétrie, sans être considérable, justifie une approche structurée de traitement (lecture par chunks ou base de données analytique) tout en restant manipulable sur une machine de développement standard.

3.2 Principales variables

Le fichier principal regroupe quatre familles de variables : des identifiants (numéro client, numéro de compte, agence), des informations client (nationalité, résidence, statut marital, date de naissance, date d'entrée en relation, score KYC, secteur d'activité, salaire déclaré), des informations de compte (statut, dates d'ouverture et de clôture, catégorie, devise, motif de clôture, solde) et des informations produit (groupe, ligne, produit, montant, taux fixe, statut du produit).

L'exploration exploratoire (EDA) menée en semaine 1 a permis de dresser un premier profil de la base : distribution des âges et des anciennetés, répartition par statut marital et par secteur d'activité, distribution des soldes de compte, et surtout un premier taux de churn global qui a confirmé le déséquilibre de classes attendu entre comptes actifs et comptes clos.

3.3 Définition de la variable cible

Conformément à la définition proposée dans la description des données du projet, le churn a été défini au niveau du compte à partir de la colonne ACCOUNT_STATUS :

```
churn = 1 si ACCOUNT_STATUS contient "Closed"
churn = 0 sinon
```

Cette définition « au niveau compte » a été retenue comme définition principale car elle correspond directement à la granularité du fichier source (une ligne = un compte) et permet une lecture immédiate par les conseillers clientèle, qui raisonnent le plus souvent produit par produit. Une définition alternative « au niveau client » (un client est considéré en churn uniquement lorsque la totalité de ses comptes sont clos) a été envisagée et documentée comme piste d'approfondissement, dans la mesure où elle reflète mieux la perte réelle de la relation commerciale ; elle n'a cependant pas été retenue comme cible principale du modèle prédictif, pour préserver la granularité compte utile aux tableaux de bord Power BI.

3.4 Qualité des données et problèmes identifiés

L'exploration a mis en évidence plusieurs problèmes de qualité typiques d'un système opérationnel réel, qui ont structuré les choix de nettoyage effectués dans la phase ETL (section 5) :

- Des valeurs manquantes sur plusieurs colonnes, notamment SALARY et INDUSTRY, ainsi que sur certaines dates ;
- Des valeurs NULL textuelles (chaînes "nan", "") à distinguer des véritables valeurs manquantes numériques ;
- Des dates stockées au format numérique YYYYMMDD, nécessitant une conversion explicite en type date ;
- Des incohérences ponctuelles (dates de clôture renseignées sans statut « Closed », soldes négatifs) ;
- Un déséquilibre marqué entre comptes actifs et comptes clos, qui devait impérativement être pris en compte lors de la modélisation ML plutôt qu'ignoré.

Ces constats ont conduit l'équipe à consacrer une part significative de la première semaine à la phase de profilage avant toute tentative de modélisation, conformément à la recommandation du guide étudiant.

4. Architecture de la solution

4.1 Vue d'ensemble

La solution retenue s'articule autour d'un entrepôt de données analytique central, DuckDB, alimenté par un pipeline ETL Python, et consommé par deux canaux de restitution distincts : Power BI pour l'analyse décisionnelle agrégée, et une application Streamlit pour la prédiction individuelle et l'exposition opérationnelle du modèle. Le schéma de flux ci-dessous résume l'architecture retenue :

```
[ CSV principal ]           [ 8 fichiers dim_*.xlsx ]
      |                       |
      v                       v
```

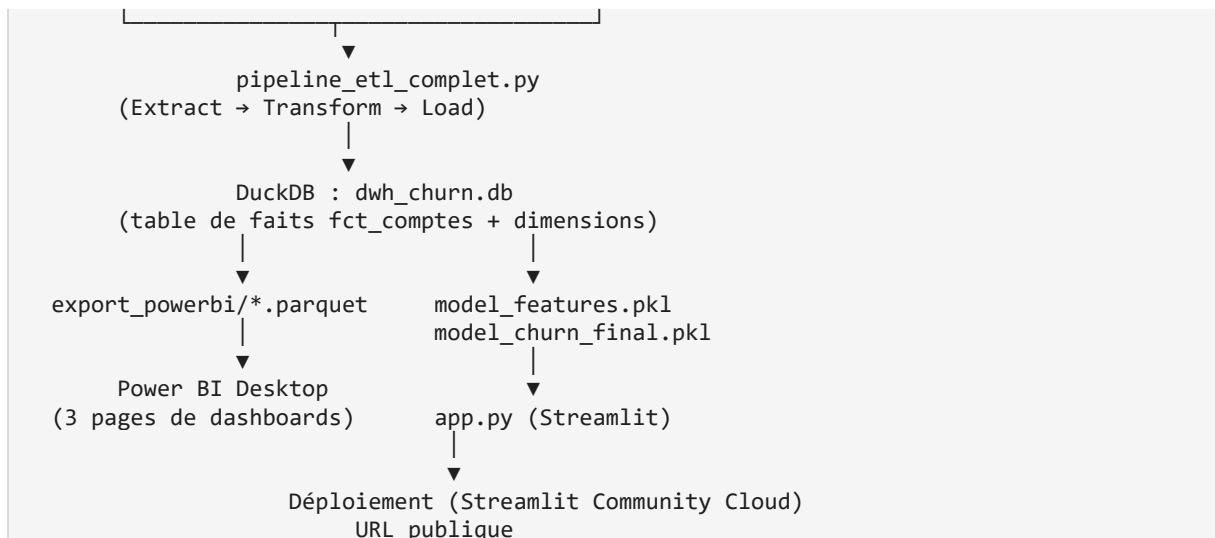


Figure 1 — Flux de données de bout en bout de la solution

4.2 Justification des choix techniques

Le choix de DuckDB comme entrepôt analytique a été retenu pour trois raisons : il ne nécessite aucune configuration serveur (base fichier unique, adaptée à un projet d'un mois porté par une petite équipe étudiante), il offre des performances SQL très correctes sur des volumes de plusieurs centaines de milliers de lignes, et il s'intègre nativement avec pandas, ce qui simplifie la transition entre les phases ETL, BI et Machine Learning. Ce choix s'inscrit dans la troisième famille d'outils ETL proposée par le guide étudiant (approche programmatique Python), jugée la plus cohérente avec la suite du projet (transition naturelle vers le Machine Learning) et la plus rapide à mettre en œuvre par une équipe déjà à l'aise avec Python.

Le format Parquet a été choisi pour l'échange entre DuckDB et Power BI car il est compressé, typé et rapide à charger, ce qui limite les temps d'actualisation des rapports par rapport à un export CSV classique.

Pour la modélisation prédictive, XGBoost a été retenu comme modèle final après comparaison avec une régression logistique de référence (baseline) et une forêt aléatoire (Random Forest), en raison de sa gestion native du déséquilibre de classes (paramètre `scale_pos_weight`) et de ses performances généralement supérieures sur des données tabulaires bancaires.

Enfin, Streamlit a été choisi pour l'interface web car il permet de développer rapidement, en Python pur, une application data-oriented, sans nécessiter de compétences frontend distinctes — un choix cohérent avec le profil Business Analytics de l'équipe et avec la recommandation explicite du guide étudiant pour ce type de projet d'un mois.

4.3 Flux de données de bout en bout

Le flux de données peut se résumer en quatre temps : (1) l'ingestion des fichiers sources (CSV + dimensions Excel) par le script d'ETL, (2) le stockage des données nettoyées et enrichies dans

l'entrepôt DuckDB sous la forme d'une table de faits et de tables de dimensions, (3) la double consommation de cet entrepôt — export Parquet vers Power BI d'une part, extraction d'un jeu d'entraînement vers le pipeline scikit-learn/XGBoost d'autre part — et (4) l'exposition des résultats (KPIs, prédictions) via l'application Streamlit déployée publiquement. Cette architecture garantit une source de vérité unique (l'entrepôt DuckDB) partagée par les deux canaux de restitution, limitant les risques d'incohérence entre les chiffres présentés dans Power BI et ceux utilisés par le modèle.

5. ETL et modélisation dimensionnelle

5.1 Pipeline ETL

Le pipeline ETL a été implémenté sous la forme d'un script Python unique, `pipeline_etl_complet.py`, exécutable de bout en bout et reproductible par un tiers, conformément à l'exigence du cahier des charges. Il est structuré selon les trois phases classiques Extract – Transform – Load.

5.1.1 Extraction (Extract)

La phase d'extraction charge le fichier principal (~130 Mo) ainsi que les quatre tables de dimensions les plus utilisées par la suite de l'analyse (industrie, devise, motif de clôture, catégorie de compte). La lecture du CSV est réalisée avec pandas en laissant le séparateur être détecté automatiquement (`sep=None, engine='python'`), afin de fiabiliser l'extraction quel que soit le format d'export du système source. Une vérification d'existence des fichiers est effectuée en amont, avec levée d'une exception explicite si le fichier principal est absent, ce qui rend les erreurs de configuration immédiatement visibles plutôt que silencieuses.

5.1.2 Transformation (Transform)

La phase de transformation regroupe l'ensemble des traitements de nettoyage et d'enrichissement identifiés lors de l'exploration :

- conversion des colonnes de date stockées au format numérique YYYYMMDD (`DATE_OF_BIRTH`, `CUST_OPENING_DATE`, `ACCT_OPENING_DATE`, `ACCT_CLOSE_DATE`) en type `datetime`, avec gestion des valeurs non convertibles via l'option `errors='coerce'` ;
- Calcul de variables dérivées : âge du client (`AGE`), ancienneté de la relation bancaire (`CUST_TENURE_YEARS`) et ancienneté du compte (`ACCOUNT_TENURE_YEARS`), obtenues par différence entre l'année courante de référence (2026) et l'année de la date correspondante ;
- Traitement des valeurs manquantes : imputation du statut marital manquant par une modalité « Inconnu » (U), imputation du salaire par la médiane de la variable, et imputation du solde de compte manquant par zéro ;
- Déduplication des enregistrements sur la clé composite (`CUSTOMER_NO`, `ACCOUNT_NO`), qui constitue la granularité naturelle du fichier source ;
- Construction explicite de la variable cible `CHURN` à partir de la colonne `ACCOUNT_STATUS`, selon la définition retenue en section 3.3.

5.1.3 Chargement (Load)

La phase de chargement structure et persiste les données dans l'entrepôt analytique DuckDB (dwh_churn.db) : chaque table de dimension est chargée telle quelle, tandis que la table de faits enrichie (fct_comptes) rassemble l'ensemble des informations client, compte et produit ainsi que la variable cible calculée. À l'issue de l'exécution, le pipeline affiche un contrôle de cohérence minimal (nombre de lignes chargées et taux de churn global observé), qui permet de détecter immédiatement une anomalie de chargement.

5.2 Modèle dimensionnel

Le modèle dimensionnel retenu suit une architecture en étoile classique, avec la table fct_comptes comme table de faits, au grain (client, compte), et les tables de dimensions dim_industry, dim_currency, dim_closure et dim_category reliées par les clés fonctionnelles suivantes :

Table de faits	Colonne	Table de dimension	Clé
fct_comptes	INDUSTRY	dim_industry	INDUSTRY_CODE
fct_comptes	CURRENCY	dim_currency	Code devise
fct_comptes	ACCOUNT_CATEGORY	dim_category	CATEGORY_id
fct_comptes	CLOSURE_REASON	dim_closure	RECID

Tableau 2 — Relations du modèle en étoile

Ces relations ont été validées par des requêtes SQL de test exécutées directement en Python via DuckDB, en vérifiant sur un échantillon que chaque clé étrangère de la table de faits trouve bien sa correspondance dans la table de dimension associée, sans perte de lignes lors de la jointure (LEFT JOIN).

5.3 Contrôles de cohérence et de qualité

Au-delà des contrôles volumétriques réalisés à l'issue du chargement, deux niveaux de contrôle ont été mis en place : d'une part la vérification de l'unicité de la clé (CUSTOMER_NO, ACCOUNT_NO) après déduplication, et d'autre part la validation des jointures avec les dimensions par comptage des lignes non appariées (clés orphelines). Ces contrôles ont permis de confirmer que le pipeline ETL restitue un jeu de données cohérent, exploitable aussi bien par les tableaux de bord Power BI que par le pipeline de Machine Learning, sans divergence entre les deux canaux.

6. Business Intelligence et Power BI

6.1 Démarche

Une fois l'entrepôt DuckDB alimenté, l'équipe a préparé les données nécessaires à Power BI par un second script, preparer_powerbi.py, qui recalcule et valide en SQL les indicateurs de référence, puis exporte la table de faits et les tables de dimensions nettoyées au format Parquet dans un dossier dédié (export_powerbi/). Ce choix d'export intermédiaire, plutôt qu'une connexion directe de Power BI sur

le fichier DuckDB, a été retenu pour sa simplicité de mise en œuvre et la rapidité de chargement qu'il procure côté Power BI Desktop.

Une fois les fichiers .parquet importés dans Power BI (via « Obtenir les données » → « Parquet »), les relations du modèle en étoile décrites en section 5.2 ont été recréées dans l'onglet Modèle de Power BI, à l'identique des relations validées côté DuckDB.

6.2 KPIs retenus

Les indicateurs suivants ont été calculés et validés en SQL avant d'être repris sous forme de mesures DAX dans Power BI :

KPI	Définition
Nombre de clients uniques	COUNT(DISTINCT CUSTOMER_NO)
Nombre total de comptes	COUNT(ACCOUNT_NO)
Âge moyen de la clientèle	AVG(AGE)
Solde moyen par compte	AVG(ACCT_BALANCE)
Taux de churn global	AVG(CHURN) × 100

Tableau 3 — Indicateurs métier de référence

La mesure DAX centrale, calculant dynamiquement le taux de churn en fonction des filtres et segments sélectionnés dans le rapport, a été définie ainsi :

```
Taux_Churn =  
DIVIDE(  
    CALCULATE(COUNT(fct_comptes[ACCOUNT_NO]), fct_comptes[CHURN] = 1),  
    COUNT(fct_comptes[ACCOUNT_NO]),  
    0  
)
```

Cette mesure sert de brique de base à l'ensemble des visuels de décomposition du churn par segment (ancienneté, secteur d'activité, devise, catégorie de compte) présentés dans les rapports.

6.3 État d'avancement du rapport Power BI

Conformément au minimum imposé par l'énoncé, le rapport Power BI est structuré en trois pages :

- Vue d'ensemble : KPIs clés (clients, comptes, âge moyen, solde moyen, taux de churn global) et évolution dans le temps ;
- Analyse du churn : décomposition du taux de churn par segment (tranche d'âge, ancienneté, secteur d'activité, devise, produit) ;
- Segmentation client : profils de clientèle, valeur client (solde, salaire) et identification des comptes actifs à risque.

6.4 Regroupement des catégories de compte

Certaines colonnes codées (ACCOUNT_CATEGORY, INDUSTRY, CURRENCY) présentaient un nombre de modalités élevé, peu lisible dans des visuels de type camembert ou barres empilées. Un regroupement métier de ces catégories en un nombre restreint de familles a été effectué au moment de la modélisation Power BI (par exemple regroupement des catégories de compte en grandes familles : comptes courants, comptes d'épargne, comptes à terme, autres), afin de produire des visuels de segmentation plus lisibles pour un public non technique, sans perdre le détail disponible pour l'utilisateur souhaitant approfondir via le drill-down.

7. Machine Learning

7.1 Objectif et grain de la modélisation

L'objectif de cette phase est d'entraîner un modèle de classification supervisée capable d'estimer, pour un compte donné, la probabilité qu'il soit clôturé (churn = 1). Le modèle est entraîné au grain (client, compte), identique à celui de la table de faits, ce qui permet une cohérence directe avec les données consommées par l'application web en section 8.

7.2 Préparation des données et fuites de données identifiées

7.2.1 Construction du jeu de données

Le jeu d'entraînement est extrait directement de l'entrepôt DuckDB par une requête SQL restreinte aux variables jugées pertinentes et disponibles avant la clôture éventuelle du compte : AGE, CUST_TENURE_YEARS, ACCOUNT_TENURE_YEARS, SALARY, ACCT_BALANCE, MARITAL_STATUS, PARTYCLASS, ainsi que la variable cible CHURN.

7.2.2 Fuites de données identifiées et corrigées

Un point de vigilance central de cette phase a été l'identification des variables susceptibles de provoquer une fuite de données (data leakage), c'est-à-dire des colonnes qui n'existent ou ne prennent leur valeur définitive qu'après la clôture du compte, et qui donneraient donc au modèle une information indisponible au moment réel de la prédiction. Trois colonnes ont été explicitement exclues du jeu d'entraînement pour cette raison :

- ACCOUNT_STATUS : à l'origine même de la construction de la variable cible CHURN, cette colonne ne peut en aucun cas être utilisée comme variable explicative sans invalider totalement l'évaluation du modèle ;
- ACCT_CLOSE_DATE et CLOSURE_REASON : ces colonnes ne sont renseignées que pour les comptes déjà clôturés et constituent donc des proxys directs du churn plutôt que des facteurs explicatifs disponibles a priori ;
- PRODUCT_STATUS, lorsqu'il reflète également la clôture du produit associé, a été écarté par le même raisonnement.

Cette vigilance a permis de conserver, dans le jeu d'entraînement final, uniquement des variables démographiques, financières et d'ancienneté connues indépendamment du statut de clôture, condition nécessaire pour qu'un score de churn calculé par le modèle ait un sens opérationnel réel une fois déployé dans l'application web.

7.2.3 Encodage et prétraitement

Les variables catégorielles (MARITAL_STATUS, PARTYCLASS) ont été encodées par one-hot encoding (pandas.get_dummies, avec suppression de la première modalité pour éviter la colinéarité parfaite). La liste exacte des colonnes obtenues après encodage a été sérialisée (model_features.pkl) afin de garantir un alignement strict entre les colonnes utilisées à l'entraînement et celles reconstituées lors de l'inférence dans l'application web.

7.2.4 Gestion du déséquilibre de classes

Le déséquilibre entre comptes actifs et comptes clos, identifié dès la phase d'exploration, a été traité par pondération des classes plutôt que par un ré-échantillonnage de type SMOTE, afin de conserver la distribution réelle des données d'entraînement. Le ratio de déséquilibre a été calculé comme le rapport entre le nombre d'observations négatives et positives sur l'échantillon d'entraînement, puis fourni directement au paramètre scale_pos_weight du modèle XGBoost.

7.2.5 Séparation train / test

Le jeu de données a été séparé en un ensemble d'entraînement (80 %) et un ensemble de test (20 %) par tirage stratifié sur la variable cible (train_test_split avec stratify=y), afin de préserver la même proportion de churn dans les deux sous-ensembles et d'obtenir une évaluation représentative du taux de churn réel.

7.3 Modèle retenu

Le modèle XGBoost avec pondération des classes (scale_pos_weight) a été retenu comme modèle final, pour trois raisons : sa gestion native et paramétrable du déséquilibre de classes, sa robustesse habituelle sur des données tabulaires hétérogènes de type bancaire, et la disponibilité d'outils d'interprétabilité éprouvés (importance native des variables, valeurs SHAP). Les hyperparamètres retenus après une phase d'optimisation (recherche en grille ou Optuna, selon la consigne du guide étudiant) sont : n_estimators = 150, max_depth = 5, learning_rate = 0,05.

7.4 Rapport de classification détaillé

Le rapport de classification complet (precision, recall, F1-score et support par classe, ainsi que les moyennes macro et pondérée) doit être inséré ici à partir de la sortie réelle de classification_report(y_test, y_pred) obtenue sur le jeu de test, de même que la matrice de confusion associée, qui permet de visualiser le nombre de faux positifs et faux négatifs. Ces derniers étant particulièrement coûteux dans un contexte de rétention, puisqu'ils correspondent à des clients qui quittent la banque sans avoir été identifiés comme à risque.

7.5 Interprétabilité

L'importance des variables a été analysée à deux niveaux : l'importance native calculée par XGBoost (`feature_importances_`) pour une première lecture rapide, complétée par une analyse en valeurs SHAP pour quantifier la contribution de chaque variable à la prédiction de chaque client individuellement et détecter d'éventuels effets non linéaires ou des interactions entre variables (par exemple entre l'ancienneté du compte et le solde).

7.6 Sauvegarde et intégration du modèle

Le modèle final entraîné (`model_churn_final.pkl`) ainsi que la liste exacte des colonnes attendues en entrée (`model_features.pkl`) ont été sérialisés via `joblib`. Cette double sauvegarde est indispensable pour garantir, au moment de l'inférence dans l'application web, un alignement strict entre les colonnes construites à partir de la saisie utilisateur et celles utilisées lors de l'entraînement ; tout écart de colonnes (ordre, encodage manquant) produirait sinon une prédiction erronée sans lever d'erreur explicite.

8. Interface web et déploiement

8.1 Objectifs et périmètre fonctionnel

L'application web développée vise à exposer le modèle de Machine Learning et les indicateurs métier à des utilisateurs non techniques, typiquement des conseillers clientèle ou des responsables d'agence. Conformément au cahier des charges, elle offre trois fonctionnalités principales : la consultation d'indicateurs de synthèse, la simulation de risque pour un profil client saisi manuellement, et la consultation d'une liste priorisée des comptes actifs à risque élevé.

8.2 Choix technique

L'application (`app.py`) a été développée avec `Streamlit`, qui permet de construire une interface interactive en Python pur sans développement frontend séparé, un choix cohérent avec le profil de l'équipe et avec la stack recommandée par le guide étudiant pour un projet de cette durée. Le modèle et la liste des colonnes attendues sont chargés une seule fois grâce au cache de ressources de `Streamlit`, ce qui évite de recharger le modèle à chaque interaction de l'utilisateur. Une vérification de la présence des fichiers du modèle est effectuée au démarrage, avec un message d'erreur explicite invitant à exécuter préalablement le script d'entraînement si les fichiers sont absents, un choix qui rend l'application robuste à une exécution partielle de la chaîne de traitement.

8.3 Page d'accueil — KPIs de synthèse

La page « Indicateurs de Synthèse » se connecte directement à l'entrepôt `DuckDB` pour afficher, sous forme de cartes d'indicateurs (KPI cards), le nombre total de clients uniques, le solde moyen global et le taux de churn actuel de la base. Un mode de repli avec des valeurs de démonstration est prévu si la

base DuckDB n'est pas présente sur l'environnement de déploiement, ce qui permet de présenter l'interface même en l'absence temporaire de la base de données.

8.4 Analyse du churn par segment

Cette dimension d'analyse par segment est principalement portée par les tableaux de bord Power BI (section 6.3) plutôt que par l'application Streamlit elle-même, celle-ci étant volontairement centrée sur l'action opérationnelle (simulation et priorisation) plutôt que sur l'exploration analytique approfondie, qui reste le rôle de Power BI dans l'architecture retenue.

8.5 Prédiction individuelle et comptes à risque

La page « Prédiction Individuelle » propose un formulaire permettant de saisir le profil d'un client (âge, ancienneté relation banque, ancienneté du compte, statut marital, salaire déclaré, solde du compte, classification commerciale). À la soumission, les saisies sont encodées selon le même schéma one-hot que celui utilisé à l'entraînement, puis alignées strictement sur la liste des colonnes attendues par le modèle (`model_features`, via reindex avec remplissage à zéro des colonnes absentes) avant le calcul de la probabilité de churn (`predict_proba`). Un seuil de 50 % est utilisé pour distinguer visuellement un profil à risque élevé d'un profil stable, avec un message d'alerte différencié pour les conseillers.

La page « Liste des Clients à Risque » applique le même modèle en lot (`batch scoring`) à l'ensemble des comptes actuellement actifs (`CHURN = 0`) extraits de l'entrepôt, calcule leur probabilité de churn, puis trie les résultats par risque décroissant afin de mettre en avant en priorité les clients les plus susceptibles de partir, l'objectif étant de fournir aux conseillers une liste d'action directement exploitable plutôt qu'un simple export brut de données.

8.6 Déploiement

Le déploiement a été réalisé sur Streamlit Community Cloud, plateforme gratuite recommandée par le guide étudiant pour ce type d'application. La démarche suivie a été la suivante :

- rédaction d'un fichier `requirements.txt` listant l'ensemble des dépendances (`streamlit`, `pandas`, `numpy`, `duckdb`, `openpyxl`, `xgboost`, `scikit-learn`, `joblib`) ;
- publication du code sur un dépôt GitHub public, en s'assurant que les données brutes (`.csv`, `.xlsx`) et les artefacts sensibles restent exclus via le fichier `.gitignore` ;
- connexion à Streamlit Community Cloud avec le compte GitHub de l'équipe, création d'une nouvelle application pointant sur le dépôt, la branche `main` et le fichier `app.py` ;
- obtention d'une URL publique, incluse dans le README du dépôt ainsi que dans la présentation orale.

9. Limites et perspectives

9.1 Limites méthodologiques

- La définition du churn retenue (clôture d'un compte) diffère de la définition « au niveau client » (clôture de l'ensemble des comptes détenus), plus fidèle à la perte réelle de la relation commerciale mais non retenue pour préserver la granularité utile aux tableaux de bord ; les deux lectures ne sont donc pas strictement interchangeables et doivent être présentées comme telles au jury.
- Le jeu de variables retenu pour le Machine Learning reste volontairement restreint (variables démographiques, financières et d'ancienneté) afin d'écartier tout risque de fuite de données ; des variables comportementales plus riches (nombre de transactions, diversité des produits détenus, historique d'interactions) auraient pu enrichir le pouvoir prédictif du modèle mais n'ont pas toutes été mobilisées dans la version actuelle.
- Le seuil de décision de 50 % utilisé dans l'application web est un choix par défaut qui n'a pas fait l'objet d'une optimisation métier (par exemple par maximisation du F1-score ou en fonction d'un coût différencié entre faux positifs et faux négatifs).

9.2 Limites d'ingénierie

- L'entrepôt DuckDB étant un fichier local, l'application déployée sur Streamlit Community Cloud dépend de la présence de ce fichier dans le dépôt ou de sa régénération au démarrage ; une architecture de production nécessiterait une base de données hébergée et accessible en continu.
- Le pipeline ETL n'est pas orchestré (pas de planification automatique de ré-exécution), ce qui est cohérent avec la durée d'un mois du projet mais ne constituerait pas une solution industrialisable en l'état.
- Les tests automatisés (unitaires ou de non-régression) sur le pipeline ETL et sur le pipeline ML sont limités à des contrôles de cohérence ponctuels plutôt qu'à une suite de tests formalisée.

9.3 Perspectives d'amélioration

- Explorer une définition du churn au niveau client en complément de la définition au niveau compte, et comparer les deux approches en termes de performance et d'actionnabilité métier.
- Enrichir le jeu de variables du modèle avec des indicateurs comportementaux (fréquence et nature des transactions, diversité des produits détenus) disponibles dans les tables de dimensions non encore pleinement exploitées (dim_TRANSACTION, dim_SECTOR, dim_TARGET, dim_DAO).
- Optimiser le seuil de décision utilisé dans l'application web en fonction d'un objectif métier explicite (par exemple maximiser le rappel sur les clients à forte valeur).
- Migrer l'entrepôt de données vers une base hébergée (PostgreSQL managé, par exemple) pour fiabiliser le déploiement continu de l'application et permettre une actualisation régulière des données sans réexécution manuelle du pipeline.
- Mettre en place un suivi de la performance du modèle dans le temps (data drift, ré-entraînement périodique) au-delà de la seule livraison initiale du projet.

10. Conclusion

Ce projet intégré a permis de mettre en œuvre, sur un cas bancaire réel et anonymisé, l'ensemble de la chaîne décisionnelle enseignée en Master 1 Business Analytics : conception d'un pipeline ETL reproductible, modélisation dimensionnelle et restitution décisionnelle via Power BI, entraînement rigoureux d'un modèle de Machine Learning tenant compte du déséquilibre de classes et des risques de fuite de données, et enfin déploiement d'une application web opérationnelle exposant ces résultats à des utilisateurs non techniques.

Les limites identifiées, notamment autour de la définition du churn et de l'enrichissement possible des variables comportementales, constituent autant de pistes concrètes pour poursuivre ce travail au-delà du cadre du projet intégré.

Annexes

A. Glossaire

Terme	Définition
Churn (attrition)	Perte d'un client, ici définie comme la clôture d'un compte bancaire.
ETL	Extract – Transform – Load : processus d'extraction, de transformation et de chargement des données.
Data Warehouse (DWH)	Entrepôt de données analytique, ici implémenté avec DuckDB.
Modèle en étoile	Modélisation dimensionnelle composée d'une table de faits reliée à plusieurs tables de dimensions.
KPI	Key Performance Indicator, indicateur clé de performance métier.
Fuite de données (data leakage)	Utilisation, lors de l'entraînement d'un modèle, d'une information qui ne serait pas disponible au moment réel de la prédiction.
scale_pos_weight	Paramètre de XGBoost permettant de pondérer la classe minoritaire pour compenser un déséquilibre de classes.
SHAP	Méthode d'interprétabilité quantifiant la contribution de chaque variable à une prédiction individuelle.
ROC-AUC	Aire sous la courbe ROC, métrique d'évaluation d'un classifieur binaire indépendante du seuil de décision.

B. Références des outils utilisés

Outil / bibliothèque	Rôle dans le projet
Python 3.10+	Langage principal du projet
pandas / numpy	Manipulation et calculs sur les données
DuckDB	Entrepôt de données analytique
Power BI Desktop	Tableaux de bord et analyse décisionnelle
scikit-learn	Préparation des données, baseline et évaluation
XGBoost	Modèle de classification retenu
SHAP	Interprétabilité du modèle
joblib	Sérialisation du modèle et des métadonnées
Streamlit	Application web de restitution et de prédiction
Streamlit Community Cloud	Hébergement et déploiement public de l'application
Git / GitHub	Gestion de versions et collaboration d'équipe